POLSKO-JAPOŃSKA AKADEMIA TECHNIK KOMPUTEROWYCH

SYLABUS PRZEDMIOTU

Nazwa przedmiotu:	**Budowa i integracja systemów informatycznych**
Kod przedmiotu:	BYT
Kierunek / Profil:	Informatyka / praktyczny
Tryb studiów:	stacjonarny
Rok / Semestr:	3 / 5
Charakter:	obowiązkowy
Odpowiedzialny:	dr hab. inż. Marta Łabuda
Wersja z dnia:	08.03.2026

1. Godziny zajęć i punkty ECTS

Wykłady	Ćwiczenia	Laboratori	Z prowadzącym	Praca własna	Łącznie	ECTS
30 h	—	—	30 h	20 h	50 h	**2**

2. Forma zajęć

Forma zajęć	Sposób zaliczenia
Wykład	Egzamin

3. Cel dydaktyczny

Celem cyklu wykładów jest zapoznanie słuchaczy z podstawowymi zagadnieniami inżynierii oprogramowania, w tym z fazami rozwoju i ewolucji oprogramowania oraz metodami podwyższenia jakości oprogramowania. Wykład dotyczy podstawowych aspektów inżynierii oprogramowania i jest zorganizowany według kolejnych faz cyklu powstawania oprogramowania. Omówione są fazy: strategiczna, gromadzenia wymagań, analizy, projektowania, konstrukcji, testowania, instalacji i konserwacji a także jego wdrażanie i pielęgnacji. Wykład przedstawia też zagadnienia związane z architekturą systemów informatycznych i

dobrymi praktykami projektowania za pomocą narzędzi CASE. Znaczna waga przypisana jest dyskusji aktualnych modeli cyklu życia oprogramowania i kryteriom doboru strategii jego wytworzenia, jak też wykorzystaniu wzorców projektowych i standardowych API oraz dobrym praktykom projektowania i implementacji systemów, w szczególności: systemy rozproszone, systemy krytyczne, czasu rzeczywistego oraz wielokrotnego użytku. Omawiane na zajęciach tematy obejmują też jakość oprogramowania i jej miary, podstawowe informacje o procesie testowania oprogramowania oraz o zarządzaniu przedsięwzięciem programistycznym, zarządzaniu konfiguracją oprogramowania, implementację, a także problemy outsourcingu informatycznego. Charakteryzuje informatyczne systemy wsparcia procesów biznesowych omawiając ich najważniejsze cechy. Zajęcia są skorelowane z wykładem i projektem przedmiotu Projekt informatyczny (PRO), zapewniając kompleksowość przedstawianych zagadnień inżynierii oprogramowania i dając studentom możliwość samodzielnego, metodycznego przeprowadzenia niedużych projektów informatycznych - od zdefiniowania problemu, poprzez dobór strategii i zaplanowanie jego rozwiązania - do zaprojektowania i wykonania praktycznych elementów produktu, z użyciem wzorców i komponentów projektowych i przeprowadzeniem testów systemowych i walidacyjnych. Wzorcowym modelem jest wykorzystanie technologii obiektowej w cyklu kaskadowym, komponentowym lub zwinnie, nie są wykluczone inne strategie wytwórcze np. dla specyficznego produktu jakim są gry komputerowe.

4. Przedmioty wprowadzające

Przedmiot	Wymagane zagadnienia
Użytkowanie komputura i wstęp do systemów operacyjnych	Podstawowa wiedza o organizacji systemu komputerowego i jego architekturze
Projektowanie systemów informacyjnych	Znajomość obiektowości, znajomość zagadnień analizy i projektowania SI
Warsztat Programisty	Rozumienie potrzeb systematyzacji procesu wytwarzania oprogramowania
Relacyjne bazy danych	Podstawy języka SQL. Diagram ERD
Projekt (stowarzyszony)	Umiejętność posługiwania się notacją UML

5. Treści programowe

1. 'nr_zajec': 1, 'wyklad': 'Wykład:', 'cwiczenia': ''

2. 'nr_zajec': 2, 'wyklad': 'Przedmiot i zagadnienia inżynierii oprogramowania. Podstawowe motywacje, społeczny kontekst przedsięwzięcia. Modelowanie pojęciowe. Pojęcie metodyki. Modele cyklu życia oprogramowania', 'cwiczenia': "
3. 'nr_zajec': 3, 'wyklad': 'Przebieg i ocena fazy strategicznej. Planowanie projektu. Projekt informatyczny; podstawowe charakterystyki, pojęcia, udziałowcy projektu; cykl życia i zakres projektu. Planowanie zadań. Identyfikacja problemu; Wzbogacony wizerunek (Rich Picture). Decyzje strategiczne i wizja rozwiązania.', 'cwiczenia': "
4. 'nr_zajec': 4, 'wyklad': 'Studium Wykonalności projektu informatycznego. Cele, płaszczyzny oceny –techniczna, ekonomiczna, organizacyjna i prawna; ryzyko przedsięwzięcia. Przykłady studiów wykonalności', 'cwiczenia': "
5. 'nr_zajec': 5, 'wyklad': 'Analiza wymagań i proces inżynierii wymagań. Wymagania stawiane oprogramowaniu. Cechy dobrego wymagania. Metody pozyskiwania wymagań, user stories. Podział i klasyfikacja wymagań. Miary i formalne techniki specyfikowania wymagań.', 'cwiczenia': "
6. 'nr_zajec': 6, 'wyklad': 'Strategie i procesy prowadzenia projektów informatycznych; tradycyjne (model kaskadowy, model V, prototypowanie, przyrostowy, spiralny) i nowoczesne cykle wytwarzania oprogramowania (reuse i komponentowość), MDA, ponowna inżynieria, RUP.', 'cwiczenia': "
7. 'nr_zajec': 7, 'wyklad': 'Zwinne metodyki wytwarzania oprogramowania. Programowanie ekstremalne. SCRUM: procesy, artefakty, role. Dobór strategii prowadzenia projektu.', 'cwiczenia': "
8. 'nr_zajec': 8, 'wyklad': 'Analiza i projektowanie architektury systemów; role, procesy, techniki i produkty w podejściu obiektowym oraz agile. Wzorce analizy i projektowania; Klasyfikacja i przykłady wzorców.', 'cwiczenia': "
9. 'nr_zajec': 9, 'wyklad': 'Optymalizacja projektu dla specyfiki produktu. Projektowanie systemów rozproszonych, krytycznych, czasu rzeczywistego i wielokrotnego użytku.', 'cwiczenia': "
10. 'nr_zajec': 10, 'wyklad': 'Zarządzanie przedsięwzięciem informatycznym. Infrastruktura - dokumentacja procesu wytwarzania i produktu programistycznego, komunikacja, struktury organizacyjne. Raportowanie. Obszary zarządzania. zarządzanie zespołem. Harmonogramowanie.', 'cwiczenia': "
11. 'nr_zajec': 11, 'wyklad': 'Implementacja, integracja i wersjonowanie oprogramowania.; środowiska narzędziowe oraz technologie. Zarządzanie konfiguracją oprogramowania.', 'cwiczenia': "
12. 'nr_zajec': 12, 'wyklad': 'Pojęcie jakości oprogramowania i dróg jej osiągania. Standardy jakości i dobre praktyki jakości. Zapewnianie jakości oprogramowania. Modele i miary jakości oprogramowania. Wprowadzenie do zarządzania jakością. Inspekcja kodu i analiza bezpieczeństwa. Metody szacowania nakładów. Model COCOMO II.', 'cwiczenia': "
13. 'nr_zajec': 13, 'wyklad': 'Testowanie i walidacja oprogramowania. Rodzaje testów testy jednostkowe, integracyjne, systemowe; walidacja i testy akceptacyjne. Typowe fazy i metody testowania. Testy statyczne, funkcjonalne i strukturalne. Czynniki sukcesu i rezultaty testowania. Dokumentowanie testów. Przeglądy oprogramowania.', 'cwiczenia': "
14. 'nr_zajec': 14, 'wyklad': 'Instalacja i wdrożenie oprogramowania. Pielęgnacja oprogramowania. Analiza potrzeby wprowadzania modyfikacji. Koszty konserwacji opro-

gramowania. Kluczowe czynniki sukcesu fazy utrzymania.', 'cwiczenia': "

15. 'nr_zajec': 15, 'wyklad': 'Outsourcing w inżynierii oprogramowania.', 'cwiczenia': "
16. 'nr_zajec': 16, 'wyklad': 'Informatyczne wsparcie procesów biznesowych. Oprogramowanie klasy CRM, ERP. Architektura systemów ERP. Przykłady dostawców systemów CRM i ERP.', 'cwiczenia': "

6. Efekty kształcenia

Wiedza

- Student zna i rozumie zaawansowane pojęcia w zakresie sieci komputerowych, protokołów komunikacyjnych i mechanizmów bezpieczeństwa sieciowego, w tym firewalle, systemy IDS/IPS oraz wirtualne sieci prywatne (VPN). [K_W08]
- Student zna i rozumie zaawansowane zagadnienia relacyjnych baz danych: modelowanie ERD, normalizację, transakcje, indeksy oraz programowanie SQL w kontekście bezpieczeństwa danych. [K_W13]
- Student zna i rozumie zasady inżynierii oprogramowania, wzorce projektowe (GoF), notację UML, cykle wytwórcze (Agile/Scrum) oraz problem jakości oprogramowania i techniczny dług. [K_W14]
- Student zna i rozumie metody i techniki inżynierii wymagań, sposoby tworzenia specyfikacji funkcjonalnych i niefunkcjonalnych oraz narzędzia do modelowania wymagań (m.in. diagramy przypadków użycia). [K_W15]
- Student zna i rozumie zaawansowane metody walidacji i testowania oprogramowania: testy jednostkowe, integracyjne, systemowe i akceptacyjne, a także narzędzia do automatyzacji testów i analizy pokrycia kodu. [K_W16]

Umiejętności

- Student potrafi zaprojektować, zainstalować i administrować siecią LAN/WAN z zachowaniem zasad bezpieczeństwa informacji, skonfigurować usługi sieciowe oraz reagować na incydenty bezpieczeństwa. [K_U14]
- Student potrafi sformułować zapytania SQL, zaprojektować schemat relacyjnej bazy danych na podstawie modelu ERD oraz zarządzać transakcjami i uprawnieniami w systemie bazodanowym. [K_U18]
- Student potrafi zaplanować i zrealizować projekt programistyczny metodą obiektową z zastosowaniem wzorców projektowych, narzędzi CI/CD oraz systemów kontroli wersji (Git). [K_U19]
- Student potrafi zaplanować i przeprowadzić pełny proces testowania oprogramowania: opracować plan testów, napisać testy automatyczne (JUnit/NUnit/pytest), wykonać testy regresji i zinterpretować wyniki. [K_U22]
- Student potrafi przeprowadzić inspekcję kodu, zidentyfikować defekty i poprawić jakość oprogramowania zgodnie z przyjętymi standardami (SOLID, Clean Code). [K_U21]

Kompetencje społeczne

- Student jest gotów do pracy w zespole deweloperskim w zróżnicowanych rolach (deweloper, tester, analityk), stosując metodyki zwinne i dobre praktyki inżynierii oprogramowania. [K_K04]
- Student jest gotów do samodzielnego poszerzania wiedzy z zakresu bezpieczeństwa systemów informatycznych i nowych technologii wytwarzania oprogramowania. [K_K03]
- Student jest gotów do dostrzegania i profesjonalnego rozwiązywania problemów etycznych i prawnych związanych z bezpieczeństwem danych i odpowiedzialnością inżynierską. [K_K06]

7. Kryteria oceny

- wyklad
- cwiczenia_laboratorium

8. Metody dydaktyczne

wyklad:

- wykład z prezentacją multimedialną
- wykład konwersatoryjny (z elementami dyskusji)

9. Literatura

Podstawowa:

- 1. Krzysztof Sacha, Inżynieria Oprogramowania, PWN 2022
- 2. Ian Sommerville, Inżynieria Oprogramowania, wyd.10, PWN 2020
- 3. Max Kanat-Alexander, Zrozumieć oprogramowanie, Helion 2019
- 4. Robert C. Martin, Czysta architektura. Struktura i design oprogramowania. Przewodnik dla profesjonalistów, Helion 2022
- 5. Mike Cohn, Agile. Metodyki zwinne w planowaniu projektów, Helion 2019
- 6. Robert C. Martin, Zwinne wytwarzanie oprogramowania. Najlepsze zasady, wzorce i praktyki, Helion 2017

Uzupełniająca:

- 1. Bernd Bruegge, Allen H. Dutoit, Inżynieria oprogramowania w ujęciu obiektowym UML, wzorce projektowe i Java Helion 2011
- 2. Keeling Michael,Zostań architektem oprogramowania, PWN 2019
- 3. Piotr Gaczkowski, Adrian Ostrowski, Architektura oprogramowania bez tajemnic, Helion 20224.

- 4. Arnon Axelrod, Automatyzacja testów. Kompletny przewodnik dla testerów oprogramowaniaPWN 2022
- 5. Strony domowe do wybranych narzędzi informatycznych. Instrukcje obsługi, przykłady.

PJATK 学大科工報情本日ポーランド